

Real-world (config + logging together) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Work through in order — don't jump ahead until you've genuinely tried. Each hint now has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real codebase would actually write it). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1](#)
- [Hint 2](#)
- [Hint 3](#)
- [Hint 4](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 1 — What's the idea here?

Basic Version

This exercise is not about writing new code — you already built a config loader and a logger. Now `main.py` just needs to use both together: load the config, get a logger, then print a log message that includes something from the config.

Think about which one you should call first.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

This exercise is about **wiring**, not new logic — you already built `load_config()` and `get_logger()` separately. Now `main.py` just needs to call both, and use them together: get the config, get a logger, then log something that includes a value from config.

Think about the order: which one do you call first, and why does that order matter?

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Advanced Version

There's a real ordering tension hiding here: you want to fail fast if config is broken, so

`load_config()` has to run first, before logging is even set up. But that means a startup failure only ever gets a bare `print()` — it never goes through your nicely configured logger, because the logger doesn't exist yet at that point.

Think about the other side of the same coin, too: once startup *succeeds*, is that worth logging on its own? A production app usually logs "started up successfully, here's the config it's running with" as its very first log line — not because anything went wrong, but because when you're debugging an incident later, the first question is often "was this instance even running with the config we think it was?"

Difference between Basic, Intermediate, and Advanced: Basic says "load config, then get a logger." Intermediate explains *why* that order matters (fail fast, before anything else runs). Advanced looks at both edges of the startup sequence — what happens when config loading fails before a logger exists, and what should happen (a logged event, not silence) once it succeeds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 2 — The exact pieces

Basic Version

At the top of `main.py`, bring in both functions you already wrote:

- `load_config()` from your `config.py`
- `get_logger()` from your `logging_setup.py`

Call `load_config()` first, before anything else — if a setting is missing, you want to find out right away.

Then use an f-string to put a config value inside your log message.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

`from config import load_config` and `from logging_setup import get_logger` at the top of `main.py`. Call `load_config()` first — if it's going to fail (a missing key), you want that to happen before you've done anything else, including setting up logging.

Use an f-string to build your log message with a value from the config object: `logger.info(f"...{config.some_field}...")`.

To handle a startup failure cleanly, wrap `load_config()` in `try / except MissingConfigError` as `e:`, and on failure `print(f"Startup failed: {e}")` followed by `sys.exit(1)` — a normal Python program exits with code `1` to signal "something went wrong" to whatever ran it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Advanced Version

Three more pieces:

- `sys.exit(f"Startup failed: {e}")` — when `sys.exit()` is given a string instead of a number, Python prints that string to stderr and exits with code `1` for you. It's a one-line replacement for the `print(...)` + `sys.exit(1)` pair.
- `def main() -> None:` with `if __name__ == "__main__": main()` at the bottom — wrapping startup logic in a function that only runs when the file is executed directly, not when it's imported.
- `logger.info("startup complete", extra={"log_level": config.log_level, "event": "startup"})` — the `extra=` argument attaches structured data to a log record. It won't show up in a plain text log line unless the formatter is written to display it, but it's exactly the kind of thing a JSON log formatter or a log-aggregation tool (Datadog, ELK, CloudWatch) reads directly, without having to parse a sentence.

Difference between Basic, Intermediate, and Advanced: Basic just names which function to call first and why. Intermediate adds the actual failure-handling code around that call. Advanced tightens the failure path to one line, makes the whole file safely importable elsewhere via `main()`, and treats a successful startup as an event worth logging too — not just failures.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 3 — The plan, in pseudocode

Basic Version

```
in main.py:
    bring in load_config and get_logger

    config = load_config()      # do this first
    logger = get_logger("main")

    log a message that includes a value from config, like the log level

run the file and check the message shows up with the config value in it
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
in main.py:
    import load_config from config.py
    import get_logger from logging_setup.py
    import sys and MissingConfigError

    try:
        config = load_config()          # do this first – fail fast if something'
    except MissingConfigError as e:
        print an error message and sys.exit(1)

    logger = get_logger(__name__)

    logger.info a message that includes config.log_level (or any field you have)
```

run it, and confirm the line appears on screen with the config value inside it
run it again with a broken/missing config value, and confirm you get a clean
one-line error instead of a Python traceback

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Advanced Version

```
in main.py:
    import load_config, get_logger, sys, MissingConfigError

    define main() -> None:
        try:
            config = load_config()
        except MissingConfigError as e:
            sys.exit(f"Startup failed: {e}")    # one line instead of print + sys.

        logger = get_logger(__name__)
        logger.info("startup complete", extra with log_level and an "event" tag)

    if this file is run directly (__name__ == "__main__"):
        call main()
```

Difference between Basic, Intermediate, and Advanced: Basic's pseudocode never fails on purpose — it assumes config always loads. Intermediate adds the failure path explicitly, with a `try / except` around the one call that can fail. Advanced tightens the failure path to one line, wraps the whole thing in `main()` so the file is safely importable elsewhere, and treats a *successful* startup as an event worth logging too, with structured data attached instead of just a sentence.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 4 — Almost the whole thing

Basic Version

```
# config_logging_wiring_practice.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger("main")

logger.info("Loaded config with log level: " + config.log_level)
```

Run this file (`python main.py`) and check the line prints.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
# config_logging_wiring_practice.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

try:
    config = load_config()
except MissingConfigError as e:
    print(f"Startup failed: {e}")
    sys.exit(1)

logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Run this file directly (`python main.py`) and confirm the line prints. Then break your config on purpose (delete a required value) and confirm you get the clean one-line "Startup failed: ..." message instead of a traceback.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Advanced Version

```
# config_logging_wiring_practice.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

def main() -> None:
    try:
        config = load_config()
    except MissingConfigError as e:
        sys.exit(f"Startup failed: {e}")

    logger = get_logger(__name__)
    # log a "startup complete" message here, with extra={...} carrying
    # config.log_level and something like "event": "startup"

if __name__ == "__main__":
    main()
```

Fill in the missing `logger.info(...)` call yourself, then compare all 3 of your finished versions against the [Solution](#) — its Advanced version also explains exactly what `extra=` does and doesn't change about what you see printed.

Difference between Basic, Intermediate, and Advanced: same 2 function calls (`load_config()` , `get_logger()`) at 3 completeness levels — Basic proves the wiring works, Intermediate adds a clean failure path so a broken config doesn't crash with a traceback, Advanced wraps startup in `main()` (safe to import elsewhere) and logs the startup event itself with structured data, not just a sentence.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Full solution: [Show me the solution](#)